Scope 1

Project Overview
This script automates the process of connecting to a remote Ubuntu server from a local Linux machine, executing a WHOIS lookup anonymously, and capturing network traffic for security analysis.

SECTION 1: Installation and Anonymity Check

Step 1: Package Installation

The script checks if required packages are installed on the local Linux machine:

nipe - Tor-based anonymity tool
whois - Domain/IP information lookup tool
sshpass - Non-interactive SSH password authentication
ftp - File Transfer Protocol client
tcpdump - Network packet capture tool
geoip-bin - IP geolocation database

```
Starting SUDO APT UPDATE...
[sudo] password for kali:
Get:1 http://mirror.freedif.org/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirror.freedif.org/kali kali-rolling/main amd64 Packages [21.0 MB]
Get:3 http://mirror.freedif.org/kali kali-rolling/main amd64 Contents (deb) [52.7
Get:4 http://mirror.freedif.org/kali kali-rolling/contrib amd64 Packages [115 kB]
Get:5 http://mirror.freedif.org/kali kali-rolling/contrib amd64 Contents (deb) [26
Get:6 http://mirror.freedif.org/kali kali-rolling/non-free amd64 Packages [187 kB]
Get:7 http://mirror.freedif.org/kali kali-rolling/non-free amd64 Contents (deb) [8
Get:8 http://mirror.freedif.org/kali kali-rolling/non-free-firmware amd64 Packages
Get:9 http://mirror.freedif.org/kali kali-rolling/non-free-firmware amd64 Contents
Fetched 75.2 MB in 9s (8,579 kB/s)
1253 packages can be upgraded. Run 'apt list --upgradable' to see them.
[i] Checking required packages...
[!] Nipe not found. Installing...
[2025-12-02 05:03:47] Installing Nipe
Cloning into 'nipe'...
remote: Enumerating objects: 2051, done.
remote: Counting objects: 100% (73/73), done.
remote: Compressing objects: 100% (49/49), done.
remote: Total 2051 (delta 43), reused 26 (delta 24), pack-reused 1978 (from 3)
Receiving objects: 100% (2051/2051), 334.32 KiB | 11.14 MiB/s, done.
Resolving deltas: 100% (1047/1047), done.
./remote_automation(15).sh: line 59: cpanm: command not found
Loading internal logger. Log::Log4perl recommended for better logging

CPAN.pm requires configuration, but most of it can be done automatically.
If you answer 'no' below, you will enter an interactive dialog for each
configuration option instead.

Would you like to configure as much as possible automatically? [yes] yes
Fetching with HTTP::Tiny:
https://cpan.org/authors/01mailrc.txt.gz
Reading '/root/.cpan/sources/authors/01mailrc.txt.gz'
.....................................................................DONE
```

If any package is missing, the script automatically installs it using apt-get. All required tools must be available for the script to use.

```
[√] Nipe installed
[√] whois is already installed
[2025-12-02 05:06:09] whois is already installed
[!] sshpass not found. Installing...
[2025-12-02 05:06:09] Installing sshpass
[√] sshpass installed successfully
[2025-12-02 05:06:13] sshpass installed successfully
[√] ftp is already installed
[2025-12-02 05:06:13] ftp is already installed
[√] tcpdump is already installed
[2025-12-02 05:06:13] tcpdump is already installed
[!] geoiplookup not found. Installing geoip-bin...
[2025-12-02 05:06:13] Installing geoip-bin
[√] geoip-bin installed successfully
[2025-12-02 05:06:17] geoip-bin installed successfully
```

Step 2: Anonymity Verification

The script checks if the network connection is anonymous using Nipe (Tor).

Process:

- Checks Nipe status
- If not anonymous, the script automatically:
    - Stops Nipe
    - Restarts Nipe
    - Waits 30 seconds for Tor connection to establish
    - Retries up to 500 times

The script will not proceed until an anonymous connection is established.

```
[i] Checking network anonymity...
[2025-12-02 05:06:17] Checking network anonymity
[i] Starting Nipe...
[2025-12-02 05:06:17] Starting Nipe (attempt 1)
[i] Anonymity check attempt 1 of 500...
[2025-12-02 05:06:17] Anonymity check attempt 1
[!] Network connection is NOT anonymous (Attempt 1)
[2025-12-02 05:06:17] Network is not anonymous on attempt 1
[i] Restarting Nipe...
[i] Starting Nipe...
[2025-12-02 05:06:32] Restarting Nipe (attempt 1)
[i] Waiting for Tor connection to establish (30 seconds)...
[i] Anonymity check attempt 2 of 500...
[2025-12-02 05:07:04] Anonymity check attempt 2
[√] Network connection is anonymous!
[2025-12-02 05:07:05] Network is anonymous
[√] Spoofed Country: DE, Germany
[√] Spoofed IP: 185.220.101.33
[2025-12-02 05:07:06] Spoofed country: DE, Germany
[2025-12-02 05:07:06] Spoofed IP: 185.220.101.33
```

Step 3: Display Spoofed Location

Once anonymous, the script displays:

- Spoofed public IP address (obtained via curl ifconfig.io)
- Spoofed country (obtained via geoiplookup)

SECTION 2: Remote Server Connection and Operations

Step 4: SSH Credentials Input

The script prompts the user to enter:

- Remote server IP address
- Remote server username
- Remote server password (hidden while typing)

```
[i] Enter remote server credentials:
Remote server IP: 192.168.152.130
Remote server username: tc
Remote server password:
```

Step 5: SSH Connection Test

The script attempts to connect to the remote Ubuntu server via SSH using sshpass. If successful, a SSH connection is established. If failed, the script exits with error.

```
[i] Testing SSH connection to remote server...
[2025-12-02 05:12:01] Testing SSH connection to 192.168.152.130
[√] SSH connection successful
[2025-12-02 05:12:02] SSH connection established
```

Step 6: Display Remote Server Information

Once connected, the script gathers and displays:

- Remote server's public IP address
- Remote server's country location
- Remote server uptime

```
[i] Gathering remote server information...

[√] Remote Server Details:
  IP Address: 103.252.200.165
  Country:
  Uptime: up 3 hours, 37 minutes

[2025-12-02 05:12:06] Remote server IP: 103.252.200.165
[2025-12-02 05:12:06] Remote server country:
[2025-12-02 05:12:06] Remote server uptime: up 3 hours, 37 minutes
```

User verifies they are connected to the correct remote server.

Step 7: Start Network Packet Capture

The script starts tcpdump on the remote server to capture network traffic.

```
[i] Starting packet capture on remote server...
[i] Checking for tcpdump on remote server...
[i] Checking for whois on remote server...
[i] Starting tcpdump capture...
Pseudo-terminal will not be allocated because stdin is not a terminal.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-88-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Dec  2 10:12:02 AM UTC 2025
```

Process:

- Checks if tcpdump is installed on the remote server. Installs them if missing
- Starts tcpdump to capture FTP traffic
- Saves capture to a pcap file.

Step 8: Target Address Input

The script prompts the user to enter IP address or domain name for WHOIS lookup

```
[v] Remote Server Details:
  IP Address: 103.252.200.165
  Country:
  Uptime: up 0 minutes

[2025-12-03 02:45:52] Remote server IP: 103.252.200.165
[2025-12-03 02:45:52] Remote server country:
[2025-12-03 02:45:52] Remote server uptime: up 0 minutes

Enter the IP address or domain for WHOIS lookup:
```

Step 9: Execute WHOIS Lookup

The script executes the WHOIS command on the remote server.

```
[i] WHOIS Results for 8.8.8.8:
──────────────────────────────────────

#
# ARIN WHOIS data and services are subject to the Terms of Use
# available at: https://www.arin.net/resources/registry/whois/tou/
#
# If you see inaccuracies in the results, please report at
# https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
#
# Copyright 1997-2025, American Registry for Internet Numbers, Ltd.
#


NetRange:       8.8.8.0 - 8.8.8.255
CIDR:           8.8.8.0/24
NetName:        GOGL
NetHandle:      NET-8-8-8-0-2
Parent:         NET8 (NET-8-0-0-0-0)
NetType:        Direct Allocation
OriginAS:
Organization:   Google LLC (GOGL)
RegDate:        2023-12-28
Updated:        2023-12-28
Ref:            https://rdap.arin.net/registry/ip/8.8.8.0
```

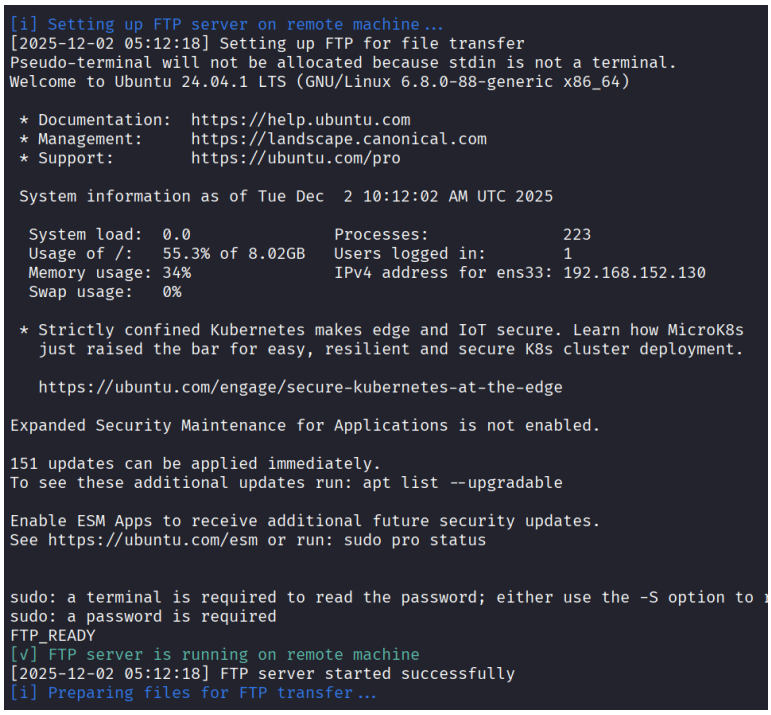
SECTION 3: Results Collection and File Transfer

Step 10: Setup FTP Server

The script configures an FTP server on the remote Ubuntu machine.

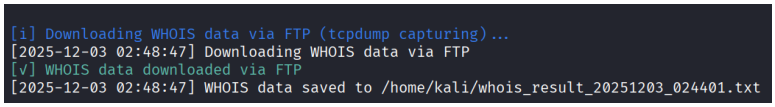
Process:

- Installs vsftpd (FTP server) if not present
- Configures vsftpd with appropriate settings
- Starts the FTP service
- Verifies FTP server is running



Step 11: Download WHOIS File via FTP

The script downloads the WHOIS data from the remote server using FTP.



Step 12: Stop Packet Capture

After FTP transfer is complete, the script stops tcpdump on the remote server.

```
[i] Allowing time for packet capture to complete...
[i] Stopping packet capture...
Pseudo-terminal will not be allocated because stdin is not a terminal.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-88-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Dec  3 07:48:42 AM UTC 2025

  System load:  0.03              Processes:              243
  Usage of /:   55.4% of 8.02GB   Users logged in:        1
  Memory usage: 30%               IPv4 address for ens33: 192.168.152.130
  Swap usage:   0%


Expanded Security Maintenance for Applications is not enabled.

151 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


tcpdump stopped
```

Step 13: Download Packet Capture File

The script downloads the PCAP file from the remote server.

```
[i] Checking packet capture results...
[v] PCAP file created on remote server (Size: 4.5K)
[2025-12-02 05:12:26] PCAP file size: 4.5K
Pseudo-terminal will not be allocated because stdin is not a terminal.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-88-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Tue Dec  2 10:12:02 AM UTC 2025

  System load:  0.0               Processes:              223
  Usage of /:   55.3% of 8.02GB   Users logged in:        1
  Memory usage: 34%               IPv4 address for ens33: 192.168.152.130
  Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

151 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


[i] Downloading packet capture file via FTP...
[v] Packet capture downloaded via FTP to /home/kali/capture_20251202_051212.pcap
[2025-12-02 05:12:26] PCAP file saved to /home/kali/capture_20251202_051212.pcap
```

Step 14: Cleanup Remote Server

The script removes all temporary files from the remote server.

Process:

- Deletes WHOIS result file
- Deletes pcap file

```
Removed /tmp/whois_20251202_051217.txt
Removed /tmp/capture_20251202_051212.pcap
Removed /tmp/tcpdump.pid
Removed /tmp/tcpdump.log
Removed ~/whois_20251202_051217.txt
Removed ~/capture_20251202_051212.pcap
Cleanup complete
[v] Remote server cleaned up
[2025-12-02 05:12:27] Remote server cleaned up
```

Final Output Summary

Upon successful completion, the script produces the following files:

1. WHOIS Result File

```
 1
 2 #
 3 # ARIN WHOIS data and services are subject to the Terms of Use
 4 # available at: https://www.arin.net/resources/registry/whois/tou/
 5 #
 6 # If you see inaccuracies in the results, please report at
 7 # https://www.arin.net/resources/registry/whois/inaccuracy_reporting/
 8 #
 9 # Copyright 1997-2025, American Registry for Internet Numbers, Ltd.
10 #
11
12
13 NetRange:       8.8.8.0 - 8.8.8.255
14 CIDR:           8.8.8.0/24
15 NetName:        GOGL
16 NetHandle:      NET-8-8-8-0-2
17 Parent:         NET8 (NET-8-0-0-0-0)
18 NetType:        Direct Allocation
19 OriginAS:
20 Organization:   Google LLC (GOGL)
21 RegDate:        2023-12-28
22 Updated:        2023-12-28
23 Ref:            https://rdap.arin.net/registry/ip/8.8.8.0
24
```

2. Packet Capture File

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.152.129	192.168.152.130	TCP	80	47248 → 21 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM TSval=2284781643 TSecr=0 WS=4
2	0.000030	192.168.152.130	192.168.152.129	TCP	80	21 → 47248 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=3400389127 TSecr=2284781643 W
3	0.000180	192.168.152.129	192.168.152.130	TCP	72	47248 → 21 [ACK] Seq=1 Ack=1 Win=129940 Len=0 TSval=2284781643 TSecr=3400389127
4	0.002467	192.168.152.130	192.168.152.129	FTP	92	Response: 220 (vsFTPd 3.0.5)
5	0.002616	192.168.152.129	192.168.152.130	TCP	72	47248 → 21 [ACK] Seq=1 Ack=21 Win=129920 Len=0 TSval=2284781645 TSecr=3400389129
6	0.002701	192.168.152.129	192.168.152.130	FTP	81	Request: USER tc
7	0.002717	192.168.152.130	192.168.152.129	TCP	72	21 → 47248 [ACK] Seq=21 Ack=10 Win=65280 Len=0 TSval=3400389130 TSecr=2284781645
8	0.002772	192.168.152.130	192.168.152.129	FTP	106	Response: 331 Please specify the password.
9	0.003037	192.168.152.129	192.168.152.130	FTP	81	Request: PASS tc
10	0.012527	192.168.152.130	192.168.152.129	FTP	95	Response: 230 Login successful.
11	0.012954	192.168.152.129	192.168.152.130	FTP	78	Request: SYST
12	0.013017	192.168.152.130	192.168.152.129	FTP	91	Response: 215 UNIX Type: L8
13	0.013196	192.168.152.129	192.168.152.130	FTP	78	Request: FEAT
14	0.013233	192.168.152.130	192.168.152.129	FTP	87	Response: 211-Features:
15	0.013270	192.168.152.130	192.168.152.129	FTP	79	Response: EPRT
16	0.013297	192.168.152.130	192.168.152.129	FTP	79	Response: EPSV
17	0.013321	192.168.152.130	192.168.152.129	FTP	79	Response: MDTM
18	0.013345	192.168.152.130	192.168.152.129	FTP	79	Response: PASV
19	0.013371	192.168.152.129	192.168.152.130	TCP	72	47248 → 21 [ACK] Seq=31 Ack=126 Win=129824 Len=0 TSval=2284781656 TSecr=3400389140
20	0.013385	192.168.152.130	192.168.152.129	FTP	86	Response: REST STREAM
21	0.013410	192.168.152.129	192.168.152.130	TCP	72	47248 → 21 [ACK] Seq=31 Ack=140 Win=129812 Len=0 TSval=2284781656 TSecr=3400389140
22	0.013422	192.168.152.130	192.168.152.129	FTP	79	Response: SIZE
23	0.013446	192.168.152.130	192.168.152.129	FTP	79	Response: TVFS

3. Audit Log File

Throughout execution, the script maintains a detailed audit log.

```
[2025-12-02 04:48:04] Script execution started
[2025-12-02 04:48:04] whois is already installed
[2025-12-02 04:48:04] sshpass is already installed
[2025-12-02 04:48:04] ftp is already installed
[2025-12-02 04:48:04] tcpdump is already installed
[2025-12-02 04:48:04] geoiplookup is already installed
[2025-12-02 04:48:04] Checking network anonymity
[2025-12-02 04:48:04] Anonymity check attempt 1
[2025-12-02 04:48:06] Network is anonymous
[2025-12-02 04:48:08] Spoofed country: NL, Netherlands
[2025-12-02 04:48:08] Spoofed IP: 45.9.148.50
[2025-12-02 04:48:20] Remote server: tc@192.168.152.130
[2025-12-02 04:48:20] Testing SSH connection to 192.168.152.130
[2025-12-02 04:48:20] SSH connection established
[2025-12-02 04:48:23] Started tcpdump on remote server (PID stored)
[2025-12-02 04:48:24] Remote server IP: 103.252.200.165
[2025-12-02 04:48:24] Remote server country:
[2025-12-02 04:48:24] Remote server uptime: up 3 hours, 14 minutes
[2025-12-02 04:48:27] Target address: 8.8.8.8
[2025-12-02 04:48:30] Started tcpdump on remote server (PID stored)
[2025-12-02 04:48:32] Executing WHOIS lookup for 8.8.8.8 on remote server
[2025-12-02 04:48:33] WHOIS lookup completed successfully
[2025-12-02 04:48:33] Setting up FTP for file transfer
[2025-12-02 04:48:34] FTP server started successfully
[2025-12-02 04:48:34] Downloading WHOIS data via FTP
[2025-12-02 04:48:34] WHOIS data saved to /home/kali/whois_result_20251202_044804.txt
[2025-12-02 04:48:39] Stopped tcpdump on remote server
[2025-12-02 04:48:41] PCAP file size: 4.5K
[2025-12-02 04:48:42] PCAP file saved to /home/kali/capture_20251202_044827.pcap
[2025-12-02 04:48:42] Remote server cleaned up
[2025-12-02 04:48:42] Script execution completed successfully
[2025-12-02 04:48:42] Files created: /home/kali/whois_result_20251202_044804.txt, /home/kali/capture_20251202_044827.pcap
```

Scope 2

File Transfer Protocol (FTP) Research

Table of Contents

1. Introduction and Purpose

1.1 Overview

File Transfer Protocol (FTP) is one of the oldest and most widely used network protocols for transferring files between computers over TCP/IP networks. Defined in RFC 959 (October 1985), FTP was designed to facilitate efficient and reliable file exchange across different systems and platforms.

1.2 Purpose and Problem Statement

Primary Purpose:
- Enable file transfer between a client and a server over a network
- Provide a standardized method for uploading and downloading files
- Allow remote file management operations (list, delete, rename, create directories)

Problems FTP Aims to Solve:

1. Platform Independence - Files need to be transferred between different operating systems (Unix, Windows, Mac)
2. Network Efficiency - Large files must be transferred reliably over potentially unreliable networks
3. Authentication - Users need to be identified before accessing files
4. Directory Navigation - Remote file systems need to be browsable and manageable
5. Transfer Modes - Different file types (text, binary) require different handling

1.3 Key Features

Authentication:
FTP provides multiple authentication mechanisms to control access to file resources. The primary method uses username and password authentication, where users must provide valid credentials before accessing the server's file system. For public file distribution scenarios, FTP supports anonymous access, allowing users to connect without personal credentials by using "anonymous" as the username and typically their email address as the password. Additionally, some FTP implementations support account-based access control, which provides an extra layer of authentication beyond the username and password, enabling administrators to define specific permissions and resource allocations for different user accounts or groups.

File Operations:
FTP supports a comprehensive set of file operations that enable complete remote file system management. Users can upload files to the server using the STOR (store) command and download files from the server using the RETR (retrieve) command. The protocol also provides file management

capabilities including the ability to delete files with the DELE command and rename files using a two-step process with RNFR (rename from) and RNTO (rename to) commands. Directory navigation and management is facilitated through commands such as LIST and NLST for viewing directory contents in detailed or simple name-only formats respectively, while MKD (make directory) and RMD (remove directory) commands allow users to create and delete directories on the remote server. These operations collectively provide users with nearly complete control over the remote file system, similar to local file management capabilities.

Transfer Modes:
FTP supports two distinct transfer modes to accommodate different file types and ensure data integrity during transmission. ASCII mode is designed for text files and automatically handles line ending conversions between different operating systems, translating between Unix line feeds (LF), Windows carriage return-line feeds (CRLF), and legacy Macintosh carriage returns (CR) as needed. Binary mode, also known as Image mode, is used for non-text files such as images, executables, and compressed archives, transmitting data byte-for-byte without any modifications or conversions. Selecting the appropriate transfer mode is critical, as using ASCII mode on binary files can corrupt the data through unintended character translations, while binary mode is considered the safer default option for modern file transfers.

Connection Types:
FTP operates using two distinct connection modes that determine how data connections are established between client and server. In Active mode, the client first connects to the server's control port and then specifies its own IP address and port number using the PORT command, after which the server initiates the data connection back to the client from its port 20. Passive mode, introduced to address firewall and NAT traversal issues, reverses this approach by having the client initiate both the control and data connections; the server responds to the client's PASV command by advertising its IP address and a dynamic port number, which the client then connects to for data transfer. Passive mode has become the preferred method in modern networks as it eliminates the common problem of firewalls blocking incoming connections to client machines, making it more reliable in contemporary networking environments.

1.4 Relevant Standards and Literature

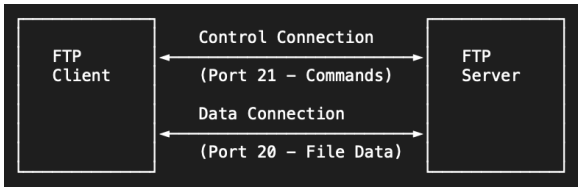
Key RFCs:
RFC 959 - File Transfer Protocol (FTP) - October 1985 (primary specification)
RFC 2228 - FTP Security Extensions - October 1997
RFC 2428 - FTP Extensions for IPv6 and NATs - September 1998
RFC 4217 - Securing FTP with TLS (FTPS) - October 2005
RFC 5797 - FTP Command and Extension Registry - March 2010

Related Protocols:
SFTP (SSH File Transfer Protocol) - RFC 4251
FTPS (FTP Secure) - FTP over SSL/TLS
TFTP (Trivial File Transfer Protocol) - RFC 1350

2. Protocol Fundamentals and Behavior

2.1 Architecture Overview

FTP uses a client-server architecture with a unique two-channel approach:



Two Separate Connections:

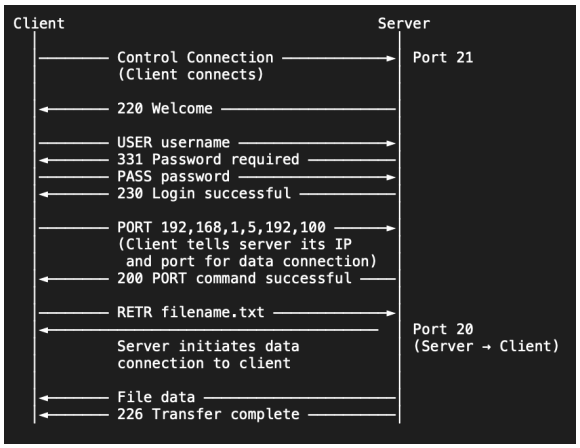
1. Control Connection (Port 21)
   - Persistent connection for commands and replies
   - Uses TCP for reliable communication
   - Remains open throughout the session
   - Carries FTP commands (USER, PASS, LIST, RETR, etc.)

2. Data Connection (Port 20 or ephemeral port)
   - Temporary connection for actual file transfer
   - Opened for each file transfer or directory listing
   - Closed after transfer completes
   - Can use different ports depending on active/passive mode

2.2 Connection Modes

2.2.1 Active Mode (PORT)

Process Flow:

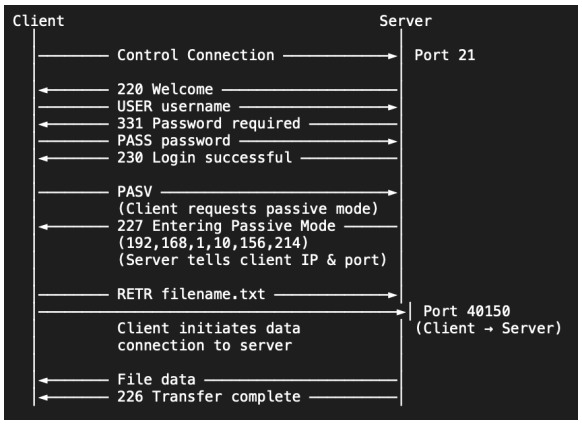


Characteristics:
Active mode operates by having the server initiate the data connection to the client, with the client first specifying its IP address and port number through the PORT command during the control session. However, this approach presents significant practical limitations in modern network environments, as the incoming connection from the server to the client is frequently blocked by firewalls and fails to work properly through Network Address Translation (NAT) devices, since the client's internal IP address may not be publicly reachable or the firewall may deny the incoming connection attempt from the external

server. This fundamental incompatibility with common network security configurations has made active mode largely obsolete in favor of passive mode, which allows the client to initiate all connections and thus bypasses these firewall and NAT traversal issues.

2.2.2 Passive Mode (PASV)

Process Flow:



Characteristics:
Passive mode resolves the firewall and NAT compatibility issues inherent in active mode by reversing the connection initiation process, with the client initiating the data connection to the server rather than waiting for an incoming connection. When the client sends the PASV command, the server responds by advertising its own IP address and an available port number (typically in the range of 1024-65535), which the client then uses to establish the data connection. This approach elegantly solves the firewall traversal problem because the client initiates all connections outbound, which are generally permitted by default firewall policies, eliminating the need for complex firewall configurations or port forwarding rules. Due to its superior compatibility with modern network security infrastructure and NAT devices, passive mode has become the de facto standard and is the most commonly used connection method in contemporary FTP implementations.

2.3 Message Exchange Process

Complete FTP Session Example

Step 1: Establish Control Connection
Client → Server: TCP SYN to port 21
Server → Client: TCP SYN-ACK
Client → Server: TCP ACK
[Control connection established]

Step 2: Server Welcome
Server → Client: 220 FTP Server Ready

Step 3: Authentication
Client → Server: USER ubuntu

Server → Client: 331 Password required for ubuntu

Client → Server: PASS secretpassword
Server → Client: 230 User logged in, proceed

Step 4: Set Transfer Mode
Client → Server: TYPE I
(I = Binary mode, A = ASCII mode)
Server → Client: 200 Type set to I

Step 5: Enter Passive Mode
Client → Server: PASV
Server → Client: 227 Entering Passive Mode (192,168,1,100,156,138)
(Server listening on 192.168.1.100, port 40074)

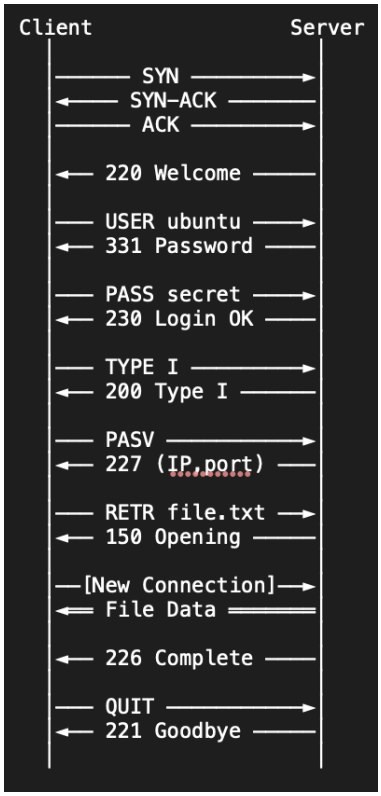
Step 6: Request File
Client → Server: RETR filename.txt
Server → Client: 150 Opening BINARY mode data connection

Step 7: Data Transfer
[Client opens new TCP connection to server's data port]
Server → Client: [Binary file data over data connection]
[Data connection closed after transfer]

Server → Client: 226 Transfer complete

Step 8: End Session
Client → Server: QUIT
Server → Client: 221 Goodbye
[Control connection closed]

2.4 Sequence Diagram



2.5 Transfer Modes

ASCII Mode (TYPE A)

- For text files
- Converts line endings between systems:
    - Unix
    - Windows
    - Mac (old)
- Risk: Corrupts binary files if used incorrectly

Binary Mode (TYPE I)

- For all non-text files (images, executables, archives)
- No conversion - byte-for-byte transfer
- Safe default for most modern transfers

3. Technical Mechanisms and Structure

3.1 Essential FTP Commands

Authentication Commands

USER <username>	- Specify username
PASS <password>	- Specify password
ACCT <account>	- Specify account (rarely used)
QUIT	- Logout

File Transfer Commands

RETR <filename>	- Retrieve (download) file
STOR <filename>	- Store (upload) file
APPE <filename>	- Append to file
DELE <filename>	- Delete file

Directory Commands

CWD <directory>	- Change working directory
CDUP	- Change to parent directory
PWD	- Print working directory
MKD <directory>	- Make directory
RMD <directory>	- Remove directory
LIST [path]	- List files (detailed)
NLST [path]	- Name list (filenames only)

Transfer Mode Commands

TYPE <type>	- Set transfer type
	A = ASCII
	I = Binary (Image)
MODE <mode>	- Set transfer mode (usually S=Stream)
STRU <structure>	- Set file structure (usually F=File)

Connection Commands

PORT <h1,h2,h3,h4,p1,p2>	- Active mode (client specifies IP/port)
PASV	- Passive mode (server specifies IP/port)

Miscellaneous Commands

NOOP	- No operation (keepalive)
SYST	- System type
STAT [path]	- Status
HELP [command]	- Help
RNFR <filename>	- Rename from
RNTO <filename>	- Rename to

3.2 Data Representation

FTP has no custom headers like HTTP. It relies on TCP for transport.

Control Connection:
The FTP control connection utilizes a simple plain text format for transmitting commands and responses between client and server, making it easily readable and debuggable by human administrators. Each command sent by the client and each response returned by the server is terminated with a carriage return and line feed sequence (\r\n, also known as CRLF), which serves as the message delimiter and ensures consistent parsing across different operating systems. This human-readable format, while beneficial for troubleshooting and educational purposes, contributes significantly to FTP's security vulnerabilities, as all commands including authentication credentials are transmitted as unencrypted text that can be easily intercepted and read by anyone monitoring the network traffic.

Data Connection:

The FTP data connection operates distinctly from the control connection by transmitting raw file content without any protocol-specific headers or formatting, sending data either in binary form or ASCII text depending on the transfer mode selected. Unlike many modern protocols that encapsulate data within structured headers, FTP's data channel carries pure file content directly through the TCP stream, with no additional protocol overhead beyond the underlying TCP/IP headers. This straightforward approach maximizes transfer efficiency but means that the data connection is ephemeral by design, opening only when a file transfer or directory listing is requested and closing immediately upon completion of the transfer. The separation of control and data channels allows FTP to maintain a persistent command session while creating and destroying data connections as needed for individual file operations.

4. Security Analysis and CIA Triad Impact

4.1 CIA Triad Overview

The CIA Triad is a fundamental security model:
Confidentiality - Information is accessible only to authorized parties
Integrity - Information remains accurate and unaltered
Availability - Information is accessible when needed

4.2 FTP's Impact on Confidentiality

CRITICAL FAILURE: No Encryption

FTP transmits ALL data in cleartext, including:
- Usernames
- Passwords
- File contents
- Directory listings
- Commands

Attack Scenarios:

1. Packet Sniffing
    a. Attacker on same network uses tcpdump/Wireshark
    b. Captures all FTP traffic
    c. Extracts credentials and files

2. Man-in-the-Middle (MITM)
    a. Attacker intercepts traffic between client and server
    b. Reads or modifies data in transit
    c. Can steal credentials, inject malicious files

3. Network Monitoring
    a. ISP or network administrator can see all FTP traffic
    b. Corporate networks can log all FTP credentials and files

Real-World Impact:
- Multiple companies suffered data breaches due to insecure FTP servers
- Credentials stolen from FTP can be reused on other services

- Sensitive documents exposed during transmission

4.3 FTP's Impact on Integrity

MODERATE FAILURE: No Built-in Integrity Checks

FTP relies solely on TCP's basic error detection

- TCP checksums can detect transmission errors
- But cannot detect intentional modification
- No cryptographic integrity verification

Vulnerabilities:

1. MITM Attacks
    a. Attacker can modify files during transfer
    b. No way to verify file hasn't been tampered with
    c. Example: Replace legitimate software with malware

2. No Authentication of Content
    a. Client cannot verify file came from legitimate server
    b. Server cannot verify uploaded file hasn't been altered

3. Directory Listing Manipulation
    a. Attacker could show fake directory contents
    b. Hide or show different files

4. Limited Protection:
    a. Some FTP clients support MD5/SHA checksums (not part of FTP protocol)
    b. Application-level integrity checks (if implemented separately)

4.4 FTP's Impact on Availability

MODERATE SUCCESS: Reliable But Vulnerable

Strengths:

1. TCP Reliability
    a. Guaranteed delivery of packets
    b. Automatic retransmission of lost packets
    c. Flow control prevents overwhelming receiver

2. Resume Capability
    a. REST command allows resuming interrupted transfers
    b. Reduces need to re-transfer entire large files

3. Mature and Stable
    a. Decades of use and refinement
    b. Well-understood failure modes
    c. Widely supported

Vulnerabilities:

1. Denial of Service (DoS)
    a. Attackers can flood server with connections
    b. Anonymous FTP especially vulnerable
    c. No authentication delay makes brute force easier

2. Firewall/NAT Issues
    a. Active mode often blocked by firewalls
    b. Multiple ports required (control + data)
    c. Complex configuration for network administrators

3. Session Hijacking
    a. Attacker can take over existing session
    b. No re-authentication during long sessions
    c. Single point of failure (if control connection dies, transfer fails)

4. Resource Exhaustion
    a. Each connection consumes server resources
    b. Multiple data connections for multiple transfers
    c. Easier to exhaust than modern protocols

4.5 Strengths of FTP

Despite security issues, FTP has some advantages:

1. Simplicity
    a. Easy to understand and implement
    b. Human-readable protocol (good for debugging)
    c. Low computational overhead

2. Universal Support
    a. Every operating system has FTP clients
    b. Built into most programming languages
    c. Web browsers support `ftp://` URLs

3. Efficiency
    a. Direct file transfer without overhead
    b. Separate data connection allows parallel transfers
    c. Resume capability saves bandwidth

4. Flexibility
    a. Anonymous access for public files
    b. Fine-grained directory permissions
    c. Support for both ASCII and binary transfers

5. Maturity
    a. Well-documented and understood

    b. Decades of real-world testing
    c. Extensive troubleshooting knowledge

4.6 Weaknesses of FTP

Critical Weaknesses:

1. No Encryption (Confidentiality)
    a. All data transmitted in cleartext
    b. Passwords easily stolen
    c. File contents exposed

2. No Integrity Verification (Integrity)
    a. Files can be modified in transit
    b. No way to detect tampering
    c. No authentication of file source

3. Poor Firewall Compatibility (Availability)
    a. Requires multiple ports
    b. Active mode often blocked
    c. Complex NAT traversal

4. Inefficient Authentication
    a. Password sent in cleartext
    b. No challenge-response mechanism
    c. Easy to brute force

5. Stateful Protocol Complexity
    a. Separate control and data connections
    b. Difficult to load balance
    c. Session state management issues

6. Vulnerable to Attacks
    a. Packet sniffing
    b. Man-in-the-middle
    c. Session hijacking
    d. Denial of service
    e. Bounce attacks (using FTP server to attack others)

SSH File Transfer Protocol (SFTP) Research

Table of Contents

1. Introduction to SFTP

1.1 Overview

SSH File Transfer Protocol (SFTP), also known as Secure File Transfer Protocol, is a network protocol that provides secure file access, transfer, and management functionality over a reliable data stream. Despite the similar name, SFTP is not simply "FTP over SSH" but rather a completely different protocol built from the ground up as an extension of the Secure Shell (SSH) protocol version 2.0. Defined in the IETF draft "SSH File Transfer Protocol" and standardized as part of the SSH protocol suite, SFTP was designed to address the fundamental security flaws present in traditional FTP while providing modern file transfer capabilities.

1.2 Purpose and Design Goals

SFTP was created with several key objectives that directly address the shortcomings of FTP. The protocol aims to provide secure file transfer capabilities with strong encryption protecting all data in transit, including authentication credentials, commands, and file contents. It incorporates robust authentication mechanisms that prevent credential theft and unauthorized access while maintaining a simple single-connection architecture that works seamlessly through firewalls and NAT devices. Additionally, SFTP includes built-in integrity checking to detect any tampering or corruption of transferred data, and it supports comprehensive file system operations beyond basic file transfer, including permission management, symbolic links, and file attribute manipulation.

1.3 Key Differences from FTP

Unlike FTP, which transmits all data in cleartext and uses separate control and data connections, SFTP encrypts everything within a single secure channel provided by SSH. This fundamental architectural difference eliminates many of FTP's security vulnerabilities and operational complexities. SFTP runs exclusively over SSH (typically port 22) rather than requiring multiple ports like FTP (ports 21, 20, and various ephemeral ports), making it inherently firewall-friendly. The protocol is binary rather than text-based, uses packet-based communication with sequence numbers for reliability, and integrates authentication directly with SSH's proven security mechanisms including public key cryptography, rather than relying on plaintext passwords.

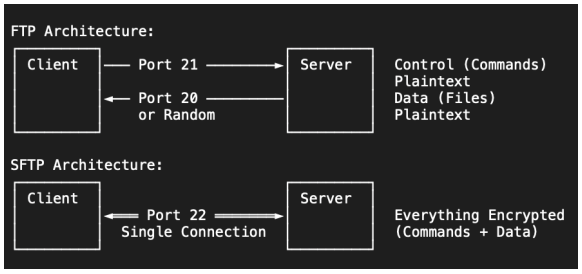
2. SFTP vs FTP: Technical Comparison

3.1 Protocol Architecture

The fundamental architectural differences between FTP and SFTP represent completely different design philosophies developed decades apart. FTP uses a dual-channel architecture with separate control connections on port 21 for commands and data connections on port 20 or ephemeral ports for actual file transfers, employing text-based commands similar to SMTP that are human-readable but insecure, and

requiring complex firewall configurations to accommodate both active and passive modes. In contrast, SFTP operates over a single SSH connection on port 22 that multiplexes all communication, uses binary packet-based communication that is efficient but not directly human-readable, and requires only a single port to be opened in firewalls, dramatically simplifying network configuration and improving compatibility with modern security infrastructure.

Architecture Comparison:



3.2 Security Mechanisms

The security differences between FTP and SFTP are stark and represent the evolution of network security understanding over several decades. FTP provides no encryption whatsoever, transmitting usernames, passwords, commands, and file contents in cleartext that can be easily intercepted by anyone with network access; it uses simple username/password authentication with no protection against credential theft; offers no integrity checking beyond basic TCP checksums that cannot detect intentional tampering; has no protection against man-in-the-middle attacks; and is vulnerable to session hijacking, packet injection, and various other attacks. SFTP, by contrast, encrypts all data using strong symmetric encryption algorithms like AES-256, ensuring complete confidentiality; supports multiple robust authentication methods including public key authentication that eliminates password transmission entirely; includes cryptographic integrity checking via MACs on every packet to detect any tampering; authenticates the server through host key verification to prevent man-in-the-middle attacks; and provides forward secrecy ensuring that compromise of long-term keys doesn't compromise past session encryption.

3.3 Authentication Comparison

The authentication mechanisms available in each protocol highlight the security gap between them. FTP's authentication is limited to username and password transmitted in plaintext over the network, making it trivial for attackers to capture credentials through packet sniffing, with anonymous FTP allowing access with no meaningful authentication at all using just "anonymous" as username and any string as password, and optional account-based authentication that still suffers from the plaintext transmission problem. SFTP offers significantly more secure options including password authentication where passwords are encrypted within the SSH tunnel before transmission, public key authentication where the client proves possession of a private key without ever transmitting it, keyboard-interactive authentication supporting multi-factor authentication and one-time passwords, certificate-based authentication using SSH certificates for scalable enterprise deployments, and GSSAPI/Kerberos integration for single sign-on in enterprise environments, with all authentication occurring within the encrypted SSH channel.|

3.4 Firewall and NAT Compatibility

One of FTP's most problematic operational issues is its poor compatibility with modern network security infrastructure, particularly firewalls and NAT devices. FTP's active mode requires the server to initiate connections back to the client, which is typically blocked by client-side firewalls and doesn't work through NAT since the client's internal IP is not publicly reachable, while passive mode helps but still requires opening a range of ephemeral ports on the server side. FTP requires special application-layer gateway (ALG) functionality in firewalls and routers to parse FTP commands and dynamically open the necessary ports, but these ALGs are often buggy or disabled for security reasons, creating reliability issues. SFTP completely eliminates these problems by using only a single well-known port (22) for all communication, with both client and server only requiring outbound connections to be permitted, which is the default for most firewalls. All data multiplexed over the single encrypted SSH channel means no dynamic port allocation or firewall inspection is needed, and the protocol works seamlessly through NAT devices without any special configuration.

4. CIA Triad Resolution

4.1 Confidentiality

SFTP transforms FTP's catastrophic confidentiality failure into robust protection through comprehensive encryption of all transmitted data. Where FTP transmitted every byte in cleartext, making usernames, passwords, file contents, directory listings, and all commands visible to any network observer, SFTP encrypts everything within an SSH tunnel using industry-standard encryption algorithms. The protocol employs AES (Advanced Encryption Standard) in various key lengths including AES-128, AES-192, and AES-256, with AES-256 providing 256-bit keys that are computationally infeasible to break with current technology, or ChaCha20-Poly1305, a modern stream cipher that is particularly efficient on systems without hardware AES acceleration. Perfect Forward Secrecy ensures that even if long-term keys are compromised in the future, past session traffic cannot be decrypted, as each session generates unique ephemeral encryption keys that are never stored.

4.2 Integrity

SFTP addresses FTP's integrity weaknesses by implementing multiple layers of cryptographic verification that detect any tampering or corruption of data. While FTP relies solely on TCP's 16-bit checksum, which can detect random transmission errors but provides no protection against intentional modification and can be easily forged by an attacker, SFTP employs Message Authentication Codes (MACs) on every single packet transmitted in both directions. These MACs, such as HMAC-SHA2-256 or HMAC-SHA2-512, use cryptographic hash functions combined with a secret key to create a verification code that is computationally infeasible to forge without knowing the key. Each packet is verified before processing, and any modification of the packet contents, whether accidental or malicious, causes the MAC verification to fail and the packet to be rejected.

4.3 Availability

SFTP significantly improves availability compared to FTP through better protocol design, enhanced resilience, and superior compatibility with modern network infrastructure. FTP's availability weaknesses stem from its vulnerability to connection exhaustion attacks where attackers open many connections to exhaust server resources, with each connection consuming significant memory and processing power, and the dual-connection model making it easier to disrupt service by attacking either control or data connections. Firewall and NAT compatibility issues frequently cause connection failures that appear as

availability problems to end users, and the protocol's lack of authentication delay makes brute force attacks efficient, potentially overwhelming servers with login attempts.

SFTP addresses these availability concerns through several mechanisms. The single-connection architecture is more efficient, reducing per-connection overhead and making resource exhaustion attacks more difficult since each client requires only one connection regardless of how many files are being transferred. SSH's built-in rate limiting and connection throttling can limit authentication attempts from a single source, preventing brute force attacks from consuming excessive resources. The protocol's native support for connection keepalives helps maintain sessions across network interruptions that would terminate FTP connections. Firewall compatibility is dramatically improved since only port 22 needs to be accessible, eliminating the complex firewall rules and port ranges required for FTP, reducing configuration errors that lead to availability problems.

Conclusion

The comparison between FTP and SFTP tells the story of network security's evolution from the trusted networks of the early internet to today's hostile environment where every connection must be secured against sophisticated threats. FTP, with its plaintext transmission and complete lack of confidentiality protection, represents a security approach that was already outdated by the 1990s and is completely unacceptable today. Its continued use is a testament to the difficulty of replacing entrenched technology rather than any remaining technical merit. SFTP, built on the robust foundation of SSH with comprehensive encryption, strong authentication, and cryptographic integrity verification, provides the security that modern data protection requires.

As we move forward in an increasingly connected and threat-filled digital landscape, the choice between FTP and SFTP is not a technical preference but a security imperative. Every organization and individual handling data across networks must ask not whether they can afford to implement strong security through protocols like SFTP, but whether they can afford not to. The lessons learned from studying both protocols—understanding what makes FTP insecure and what makes SFTP secure—provide the foundation for making informed decisions about network security that protect data, maintain privacy, and build trust in our digital systems.

The path from FTP to SFTP represents progress in our understanding of security, and continuing to use the insecure protocol when secure alternatives exist is not just technically inadvisable but ethically questionable when it exposes others' data to unnecessary risk. This conclusion is not just the end of a technical comparison but a call to action for anyone still using FTP to recognize the risks, understand the alternatives, and make the transition to secure file transfer protocols that protect the confidentiality, integrity, and availability of data in transit.

References

RFC Documents and Standards

RFC 959 - Postel, J. & Reynolds, J. (1985). File Transfer Protocol (FTP). Internet Engineering Task Force. https://tools.ietf.org/html/rfc959

RFC 4251 - Ylonen, T. & Lonvick, C. (2006). The Secure Shell (SSH) Protocol Architecture. Internet Engineering Task Force. https://tools.ietf.org/html/rfc4251

RFC 4252 - Ylonen, T. & Lonvick, C. (2006). The Secure Shell (SSH) Authentication Protocol. Internet Engineering Task Force. https://tools.ietf.org/html/rfc4252

RFC 4253 - Ylonen, T. & Lonvick, C. (2006). The Secure Shell (SSH) Transport Layer Protocol. Internet Engineering Task Force. https://tools.ietf.org/html/rfc4253

RFC 4254 - Ylonen, T. & Lonvick, C. (2006). The Secure Shell (SSH) Connection Protocol. Internet Engineering Task Force. https://tools.ietf.org/html/rfc4254

Internet-Draft - Galbraith, J., Ylonen, T., & Lehtinen, S. (2006). SSH File Transfer Protocol. IETF SECSH Working Group. https://datatracker.ietf.org/doc/html/draft-ietf-secsh-filexfer

RFC 2228 - Horowitz, M. & Lunt, S. (1997). FTP Security Extensions. Internet Engineering Task Force. https://tools.ietf.org/html/rfc2228

RFC 4217 - Ford-Hutchinson, P. (2005). Securing FTP with TLS. Internet Engineering Task Force. https://tools.ietf.org/html/rfc4217

Books and Technical References

Barrett, D. J., Silverman, R. E., & Byrnes, R. G. (2005). SSH, The Secure Shell: The Definitive Guide (2nd ed.). O'Reilly Media.

Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). UNIX Network Programming, Volume 1: The Sockets Networking API (3rd ed.). Addison-Wesley Professional.

Kozierok, C. M. (2005). The TCP/IP Guide: A Comprehensive, Illustrated Internet Protocols Reference. No Starch Press.

Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.). Pearson Education.
Kurose, J. F., & Ross, K. W. (2017). Computer Networking: A Top-Down Approach (7th ed.). Pearson.

Security and Cryptography

Ferguson, N., Schneier, B., & Kohno, T. (2010). Cryptography Engineering: Design Principles and Practical Applications. Wiley Publishing.

Schneier, B. (2015). Applied Cryptography: Protocols, Algorithms, and Source Code in C (20th Anniversary ed.). Wiley.

NIST (National Institute of Standards and Technology). (2019). Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations (SP 800-52 Rev. 2).

OWASP Foundation. (2023). OWASP Top Ten Web Application Security Risks. https://owasp.org/

Network Analysis and Tools

Wireshark Foundation. (2023). Wireshark User's Guide. https://www.wireshark.org/docs/

tcpdump/libpcap. (2023). tcpdump Documentation. https://www.tcpdump.org/

OpenSSH Project. (2023). OpenSSH Manual Pages. https://www.openssh.com/manual.html

Security Standards and Compliance

PCI Security Standards Council. (2022). Payment Card Industry Data Security Standard (PCI DSS) v4.0.

U.S. Department of Health and Human Services. (2013). HIPAA Security Rule Technical Safeguards.

European Union. (2016). General Data Protection Regulation (GDPR).

Academic Papers and Research

Rogaway, P. (2011). Evaluation of Some Blockcipher Modes of Operation. Cryptography Research and Evaluation Committees (CREC) for the Government of Japan.

Bellare, M., & Namprempre, C. (2000). Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm. In Advances in Cryptology—ASIACRYPT 2000.